



莫队算法

1: 莫队算法名字的由来

莫队算法是一个叫莫涛的国家队队长发明的算法。

2: 莫队算法可以用来解决什么问题？

莫队算法号称可以解决一切可以离线区间查询的问题。

3: 莫队算法是数据结构吗？

不是，莫队算法实际上是一种基于排序的比较巧妙的暴力算法。



数列找不同

现有一个长度为 n 的数列 A ，每个数为正整数，先给定 Q 个询问 $[l, r]$ ，问区间内的数是否互不相同。

输入：

4 2

1 2 3 2

1 3

2 4

输出：

Yes

No

$N \leq 100000$

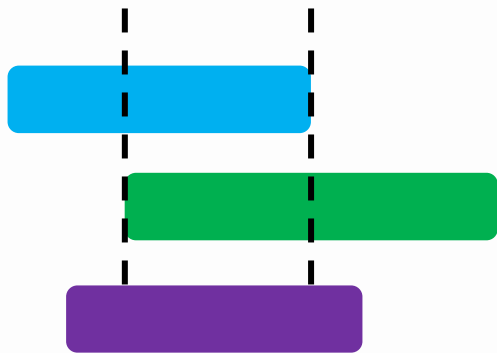
解题思路

这是一道比较简单的区间查询的题，正因为他简单，所以才麻烦，因为没有什么数据结构可以快速解决这种问题。线段树，向上合并的时候没有办法合并；平衡树 (map)，每次区间不同，平衡树需要重构，效率比暴力还低。所以很多时候，对于一些区间查询的问题，实际上并没有什么行之有效的数据结构可以解决。

那我们应该怎么办呢？貌似只有暴力！那我们能不能通过思维来优化这种暴力呢？就算是暴力，我们也是和别人不一样的暴力。

该题中，暴力求解就是每次枚举区间 $[l, r]$ ，判断区间 $[l, r]$ 中是否有重复的数据。那我们能不能少枚举一点呢？

解题思路



如上图所示，当我们枚举完第一个区间的时候 $[l_1, r_1]$ ，很明显，第一个区间和第二个区间是有交集的，交集部分实际上在前面已经计算过了，完全没有必要再枚举了。我们只需要把 $l_1 \rightarrow l_2$ ， $r_1 \rightarrow r_2$ 就可以了，这样就节省了一些时间。

我们再看下一个区间 $[l_3, r_3]$ ，也可以用同样的办法来减少枚举次数，但是还能不能再优化呢？我们会发现如果查询区间顺序为1 3 2，效率会更高，这样重复的次数会更少。

解题思路



1→2→3: 枚举次数

$$4+3+4+2+3=16$$

1→3→2: 枚举次数

$$4+1+1+2+3=11$$

所以，我们会发现如果把区间按照某一个端点从大到小排序之后再枚举区间，效果会好得多，这样至少能保证排好序的那一边做多挪动 n 次，大大减少了时间复杂度。

代码

```
while(r+1<=q[i].f)//右区间右移
{
    num[a[r+1]]++;
    if(num[a[r+1]]>1) sum++;
    r++;
}
while(r>q[i].f)//右区间左移
{
    num[a[r]]--;
    if(num[a[r]]) sum--;
    r--;
}
while(l<q[i].s)//左区间右移
{
    num[a[l]]--;
    if(num[a[l]]) sum--;
    l++;
}
```

一定要注意区间端点处是否被计算了。

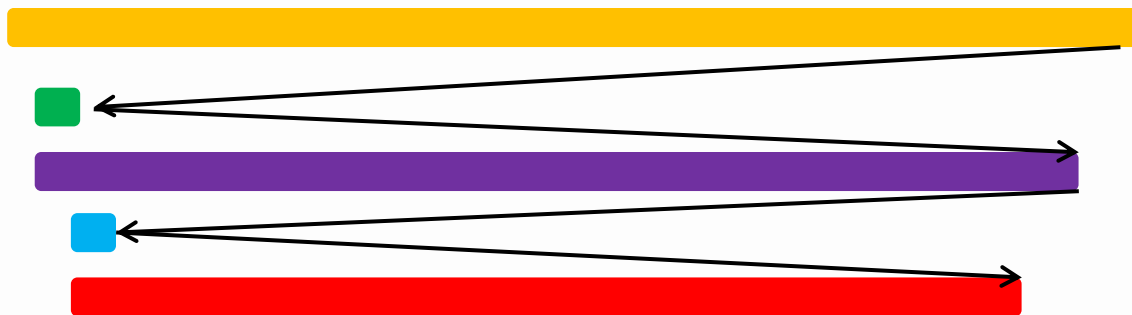
弊端

前面讲的就是莫队算法吗？当然不是，如果莫队算法真这么简单，谁都能想得到。而且前面这种算法很容易出数据把该算法卡死的。

比如 $[1, 10000]$, $[2, 2]$, $[2, 9999]$, $[3, 3]$, $[4, 9900]$ 。。。

如果按照左端点排序，你会发现右端点每一次都在飞跃。每次枚举挪动的次数相对于普通暴力减少了一半，但是效率仍然低到无法接受。

那我们要如何在原有基础上来解决这个问题呢？



莫队算法

“区间大概处于升序状态”该如何来判断呢？也就是说如果左端点相差不是很大，我们可以看做左端点相同，再按照右端点排序。

相差不是很大该如何来判断呢，我们可以把连续的一个区间的数看做是相同的，也就是把一个序列分成了很多块。是不是很熟悉，实际上这就是前面我们学习的分块的技巧，也就是说我们不再判断左端点是否相同，而是判断左端点是否在一个块中，如果左端点再一个块中，我们再按照右端点排序，这样区间排序之后效率会更高，也更符合实际要求。

这就是莫队算法的核心思想。

貌似很多题也可以用前缀和+离线树状数组也可以过。

莫队算法

第一步：排序

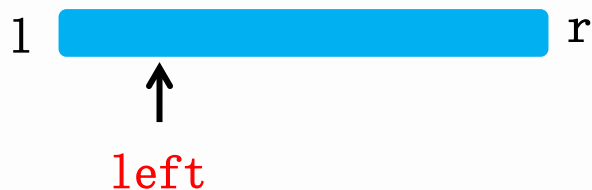
先按照左区间所在块的大小排序，如果左区间所在块相同，按照右区间大小排序。

```
bool cmp(node x,node y)
{
    if(x.l/bl==y.l/bl)//如果左端点在一个块内
        return x.r<y.r;//没必要再/bl了
    return x.l/bl<y.l/bl;
}
```

莫队算法

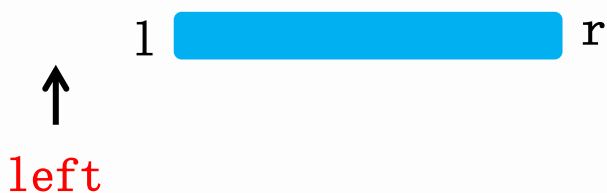
第二步：区间移动

(1) 左区间左移(因为是按照块排序，所以排序之后左区间实际大小未知)。



插入操作，插入元素 $a[1] \leftarrow a[\text{left}-1]$

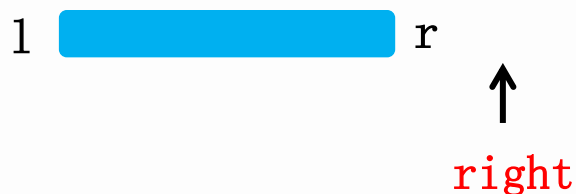
(2) 左区间右移



删除操作，删除元素 $a[\text{left}] \leftarrow a[l-1]$

莫队算法

(3) 右区间左移



删除操作，删除元素 $a[r+1] \sim a[right]$

(2) 右区间右移



插入操作，插入元素 $a[right+1] \sim a[r]$

莫队算法

第三步：判断

我们需要判断区间内是否有元素重复，可以按照桶排序的思想，把每种元素放入桶中，计算被用过的桶的数量，如果被用过桶的数量 $==$ 区间长度+1，说明没有重复，否则表示有重复。也可以记录重复次数，来一个重复元素，重复次数+1，删除一个重复元素，重复次数-1，如果最后重复次数=0，说明没有重复元素。

莫队算法的时间复杂度也是可以证明的。排序之后，左端点按块递增的，左端点移动的总体时间复杂度为 $O(n)$ ，右端点，如果按照前面那种极限情况，来回滚动的情况，那么说明相邻两个左端点不在同一个块中(如果在同一块中，不会出现来回滚动的情况)，这种情况最多 $\sqrt{n}$ 个，每次时间复杂度为 $O(n)$ ，总的时间复杂度为 $O(n^{1.5})$



练习题

洛谷 P3901 数列找不同

洛谷 P2709 小B的询问

洛谷 P1972 HH的项链

洛谷 P1494 小Z的袜子

洛谷 P2336 喵星球上的点名



带修改的莫队算法

前面我们学习的莫队算法是一种离线算法，他是把所有的询问按照某种规则排序之后再统一处理，那么很明显，他是不支持修改操作的，因为一旦数据被修改，那么修改前和修改之后查询的结果肯定是不一样的。这样就显得比较鸡肋，如果需要带修改的区间查询可以用莫队算法吗？

肯定是可以的，我们可以记录一下修改时间，即第 i 个询问是发生在第 k 个修改操作之后的。也就是说，在进行莫队算法的时候，先判断询问的时间戳和修改的时间戳是否相符，如果不符合进行时光倒流或者时光推进来保证答案的正确性。

虽然效率会低一点，但是还是要比暴力效果好很多。

数颜色

墨墨购买了一套N支彩色画笔（其中有些颜色可能相同），摆成一行，墨墨会向你发布如下指令：

1、 Q L R代表询问你从第L支画笔到第R支画笔中共有几种不同颜色的画笔。

2、 R P Col 把第P支画笔替换为颜色Col。

输入：

6 5

1 2 3 4 5 5

Q 1 4

Q 2 6

R 1 2

Q 1 4

Q 2 6

输出：

4

4

3

4

数颜色

该题是一道带修改的莫队的模板题，当然该题还有很多不同的做法。

带修改的莫队和普通莫队的区别在于他需要判断上一次所在的时间戳和当前询问所在时间戳是否相同，如果不相同，如果 $\text{now} < q[i].t$ ，那么就 $\text{now}++$ 直到相等位置，同时根据修改的情况来进行统计；如果 $\text{now} > q[i].t$ ，那么就 $\text{now}--$ ，把当前的颜色还原为以前的颜色。

带修改的莫队分块的时候最好分为 $n^{2/3}$ 块，这样做效率会达到最大，时间复杂度为 $O(n^{5/3})$ 。网上有证明，自行推导

数颜色

第一步：记录

(1) 记录查询

记录查询区间的左端点，右端点，第几次查询以及该查询处于第几次修改之后。

(2) 记录修改

记录修改的位置以及修改的颜色，这里可以不需要记录修改前的颜色，后面有一个巧妙的办法可以解决。

```
struct node
{
    int l,r,id,t;
}q[maxn]; //询问
struct point
{
    int id,color;
}re[maxn]; //修改
```

数颜色

第三步：离线处理询问

和普通莫队一样，我们先还是和以前一样的办法快速锁定区间。

```
void update1(int x)//增加
{
    if(!num[a[x]])    sum++;
    num[a[x]]++;
}
void update2(int x)//减少
{
    num[a[x]]--;
    if(!num[a[x]])    sum--;
}
while(l<q[i].l)    update2(l++);
while(l>q[i].l)    update1(--l);
while(r<q[i].r)    update1(++r);
while(r>q[i].r)    update2(r--);
```

数颜色

第四步：锁定区间之后再把时间调整到当前询问所处于的修改时间。虽然分为时光倒流和时光推进，但是实际上这两种操作可以写成一种操作。

```
void change(int x,int l,int r)
{
    if(re[x].id<=r&&re[x].id>=l)//如果修改在区间内
    {
        num[a[re[x].id]]--;
        if(!num[a[re[x].id]])
            sum--;
        if(!num[re[x].color])
            sum++;
        num[re[x].color]++;
    }
    swap(re[x].color,a[re[x].id]);//始终保证a[]是修改之后的实际颜色
}
```

数颜色

最后一个操作是带修改的莫队操作的精髓，同时也是初学者最难理解的地方。首先判断修改是否处于要查询的区间，如果不在区间之中，修改不影响答案。

对于一个位置，当前的颜色为`a[re[x].id]`，修改之后的颜色为`re[x].color`，那么前一个颜色数量减少，后一个颜色数量增加，再判断一下是否会影响颜色的种类。

最后一个swap函数是什么意思呢？

因为我们并没有记录每次颜色修改之前的颜色，所以我们要让`a`数组始终表示当前时间戳下的实际颜色，所以每次修改之后`a[re[x].id]`都要变成`re[x].color`。

那为什么`re[x].color`要变成`a[re[x]].id`呢？

因为实际操作过程中涉及修改和修改还原两种操作，而这里我们是写在一起的，对于修改之后再还原，就相当于修改之后，交换位置之后，再修改。



练习题

洛谷 P1903 数颜色

洛谷 P4396 作业

莫队算法除了前面学习的普通莫队和带修改的莫队算法之外，还有树形莫队，自行研究。主要是掌握如何在树上分块。



总结

莫队算法和分块一样都是基于区间问题的暴力算法。分块主要还是分治的思想，而莫队是基于排序减少重复的思想，理论上来说，只要一道区间问题可以转换成离线求解，那么就可以用莫队来解决，但是我们先水平有限，只能解决单点修改的莫队，对于区间修改的莫队还有一定的问题。